LINGAP: Legal Intake Navigator for Guidance and Assistance Program

TEAM PLASTICS

Buton, Kylian
Pulbosa, Mariel Sofia
Rebollido, Pauline Mae

Blog

In partial fulfillment of the requirements in CS 191

CS 191 - WFR1
Ligaya Leah Figueroa, PhD

Web App Requirements Model

A. Content Model

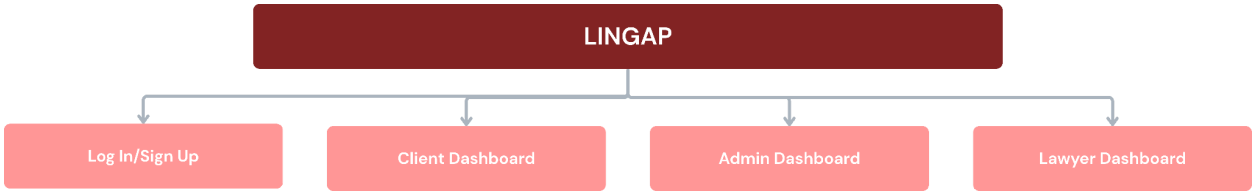


Figure 1.1 High Level Data Tree of LINGAP

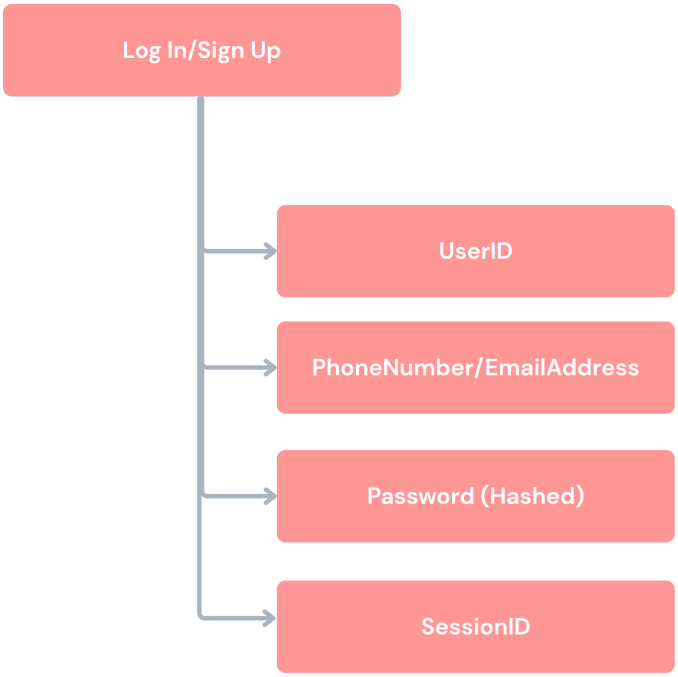


Figure 1.2 Data Tree of the Log in/Sign Up Branch

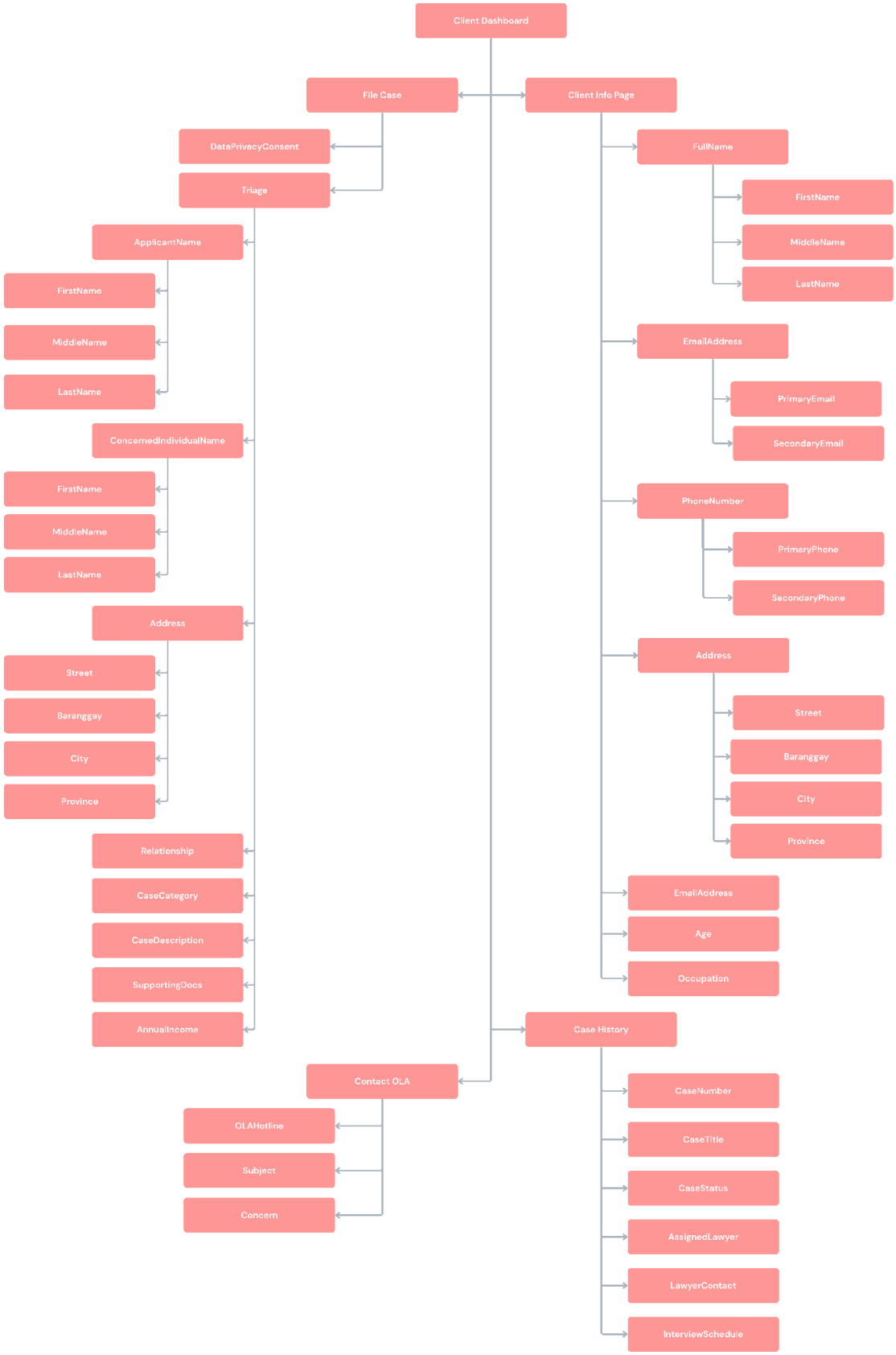


Figure 1.3 Data Tree of Client Dashboard Branch



Figure 1.4 Data Tree of Admin Dashboard Branch



Figure 1.5 Data Tree of Lawyer Dashboard Branch

B. Interaction Model

1. Actors and Goals

   Initiating Actors

   - Client - File Case, View Case Status, Contact OLA, Update User Information, View Assigned Lawyer Contact Info
   - Admin - File Case for Client, Update Case Status, Process Queue, Review Cases
   - Lawyer - Take Unassigned Cases, Schedule Client Interview, View Assigned Cases, View Client Contact Info

   Participating Actors

   - SMS/Email Gateway - Send Notifications
   - Document Manager - Store and Retrieve Documents
   - Ticketing System - Queue Cases, Assign Case Number

2. Use Cases
   - Contact OLA – Allows a client to contact the office for inquiries or assistance.
   - File Case – Allows a client to submit a request to file a legal case.
   - View Status – Allows a client to view the current status of their case.
   - View Assigned Lawyer Contact Information – Allows a client to view their assigned lawyer's contact details.
   - File Case for Client – Allows admin to file a case on behalf of a client.
   - Update Case Status – Allows an admin to update the status of a case.
   - Process Queue – Allows an admin to process cases waiting in the queue.
   - Review Cases – Allows an admin to review submitted cases.
   - Take Unassigned Cases – Allows a lawyer to take an available unassigned case.
   - Schedule Client Interview – Allows a lawyer to schedule a client interview.
   - View Assigned Cases – Allows a lawyer to view cases assigned to them.
   - View Client Contact Information – Allows lawyers to view client contact details.
   - Submit Documents – Allows a client to upload case documents.
   - Agree to Data Privacy Consent – Allows a client to provide consent for data collection and processing.
   - Answer Triage – Allows a client to answer questions used to assess their case.
   - Assign Case Number – Allows the ticketing system to assign a unique case number.
   - Queue Case – Places a submitted case into the appropriate processing queue.
   - Send Notification – Sends case-related notifications to relevant users.
   - Store Documents – Saves submitted documents in the system.
   - Retrieve Documents – Retrieves stored documents for authorized users.
   - View Documents – Allows authorized users to view case documents.

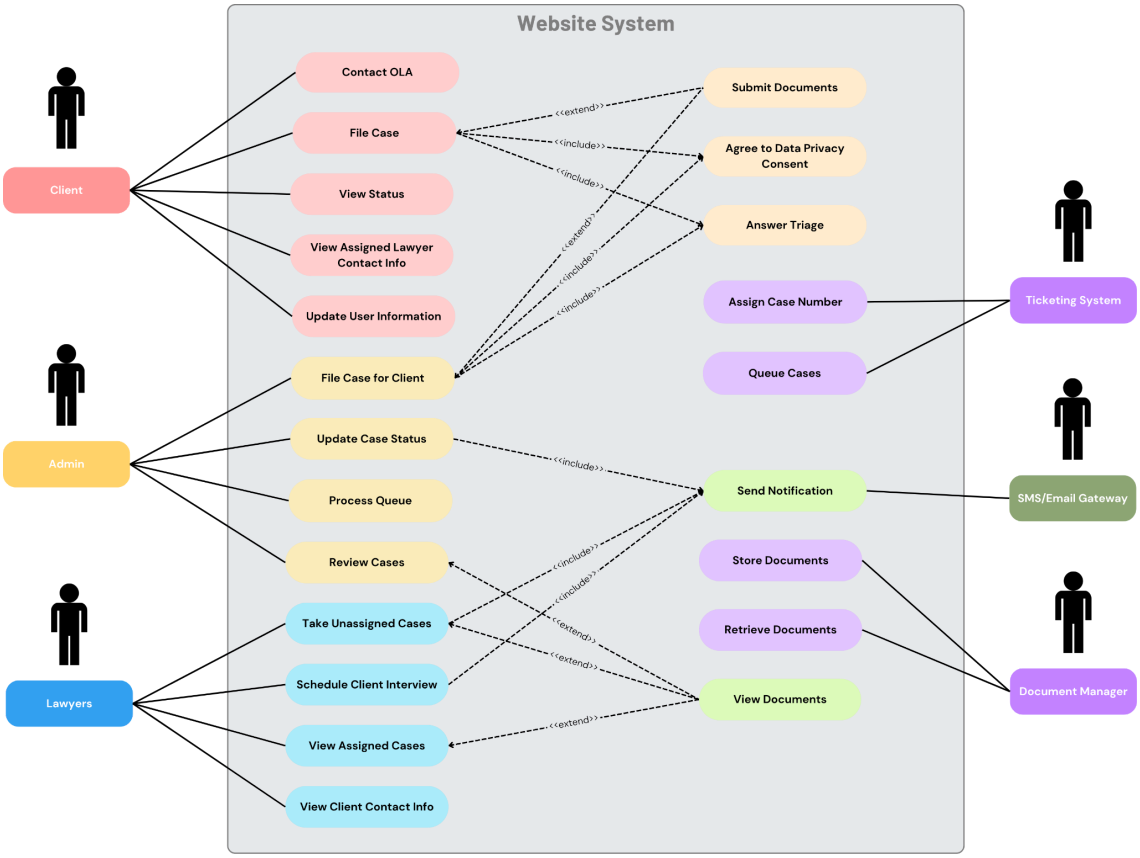


Figure 1.6 Use Case Diagram for LINGAP

3. System Sequence Diagrams

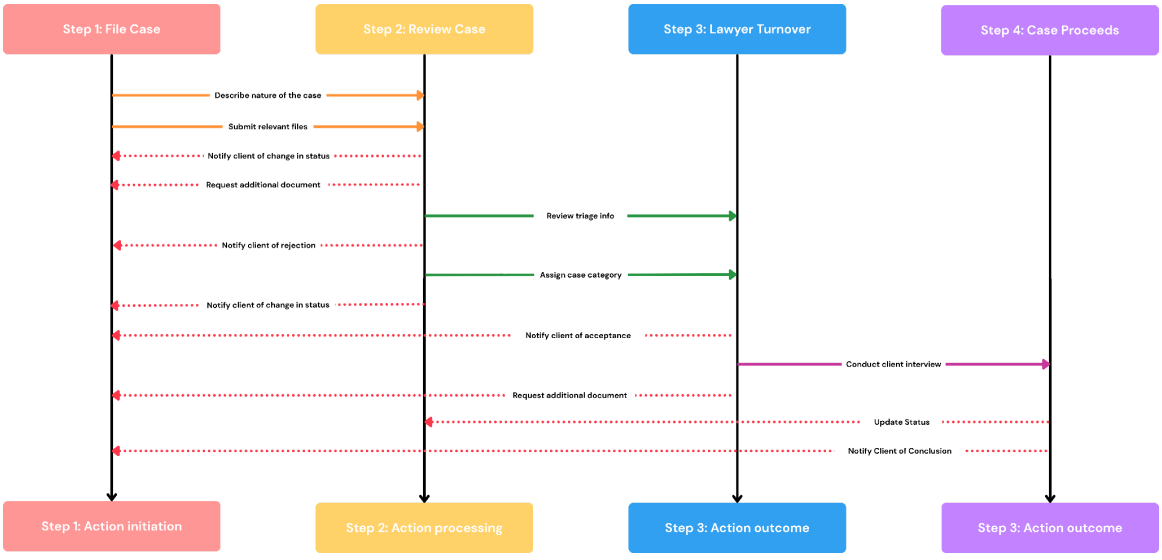


Figure 1.7 Sequence Diagram for LINGAP

C. Configuration Model

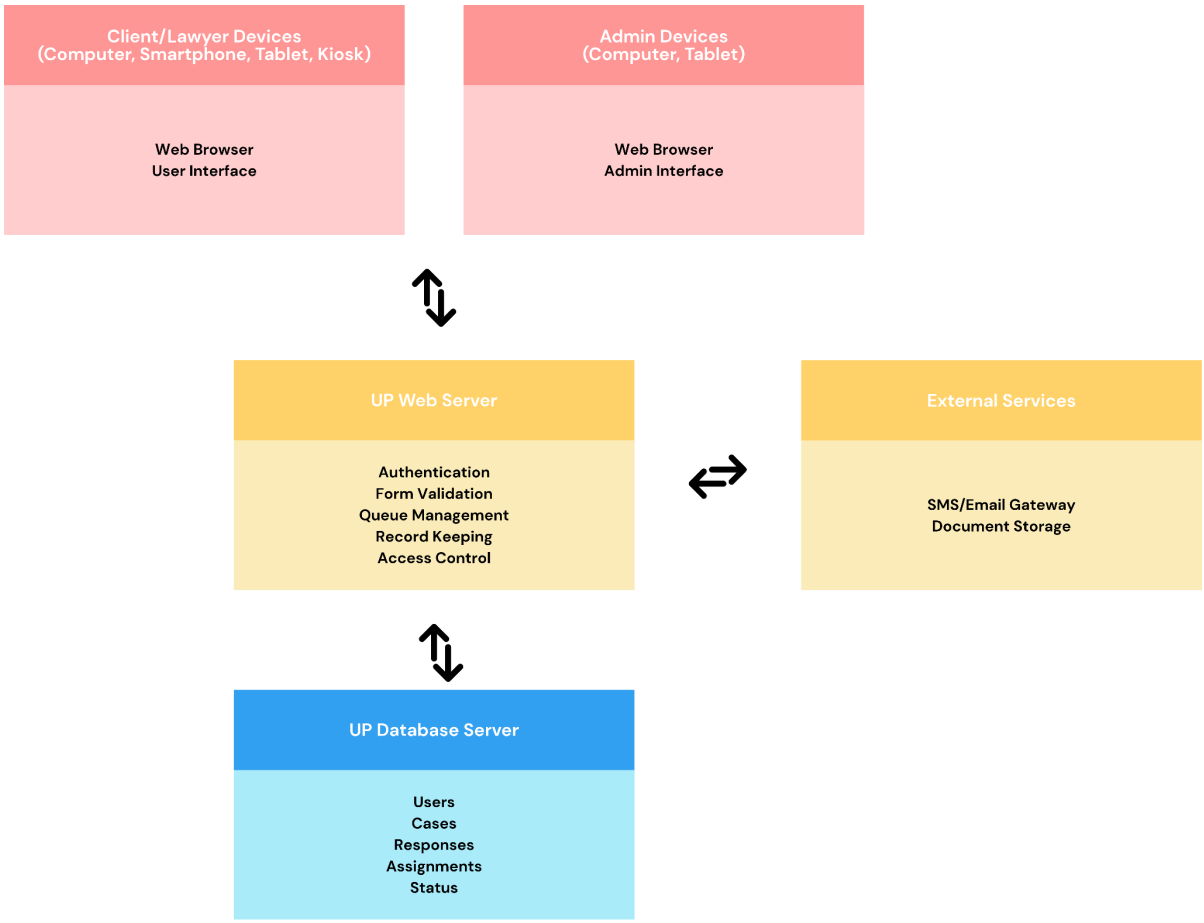


Figure 1.8 Configuration Model for LINGAP

Web App Architecture and Design

A. Architecture Design

I. Architectural Overview

LINGAP adopts a layered architecture incorporating the Model-View-Controller (MVC) pattern and a centralized data repository. The architecture separates the system into three layers: presentation, application, and data.

The presentation layer provides the public applicant portal, staff-assisted kiosk, and OLA administrative dashboard. The application layer handles eligibility screening, appointment scheduling, document management, case triage, and notifications. The data layer maintains applicant records, applications, appointments, legal matters, and supporting documents.

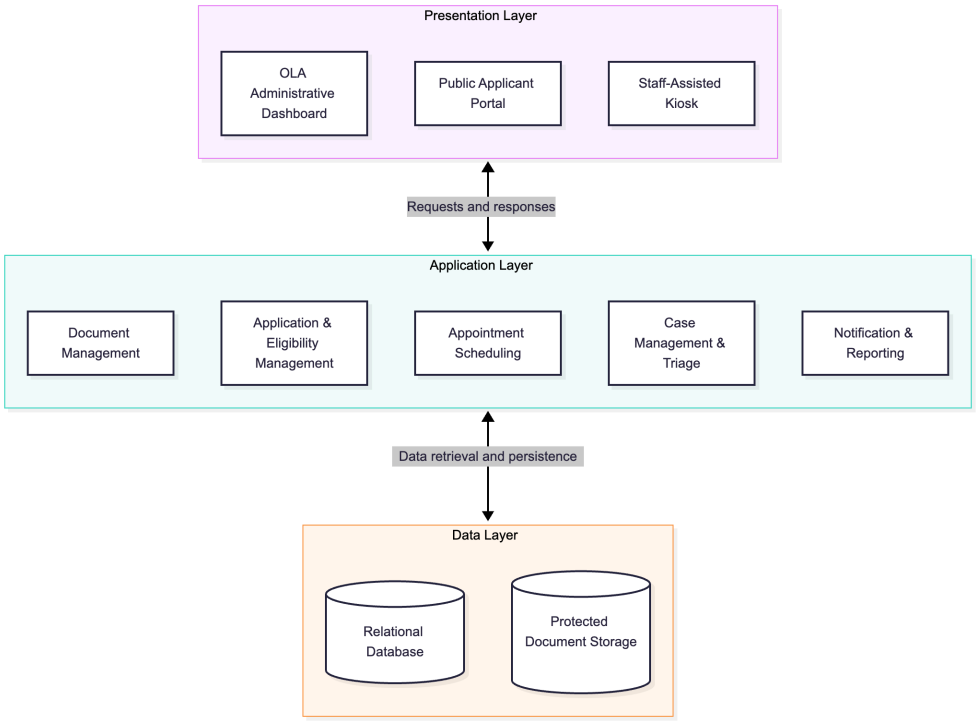


Figure 2.1. Layered architecture of the LINGAP web application.

II. Architectural Styles

LINGAP employs three complementary architectural styles.

Architectural Style	Application in LINGAP
Layered Architecture	Separates user interfaces, business logic, and data persistence to improve maintainability.
Model-View-Controller (MVC)	Separates application data and operations (Model), interface presentation (View), and request handling (Controller).
Data-Centered Architecture	Maintains a centralized repository shared by application processing, scheduling, case management, and reporting components.

The application follows a predominantly hierarchical content structure. The public portal uses a linear workflow for pre-screening, while the administrative portal provides navigation to operational modules.

III. Quality Requirements

The architecture addresses the following quality attributes:

Attribute	Design Response
Security	Role-based access control, encrypted communication, and protected document storage.
Availability	Centralized hosting, monitoring, and backups to support the 99% office-hours uptime target.
Scalability	Modular components and efficient database access.
Performance	Server-side processing and indexed database queries.
Maintainability	Separation of concerns and well-defined component interfaces.
Usability	Responsive bilingual interfaces, guided forms, and staff-assisted access.
Privacy	Explicit consent, data minimization, and 30-day retention for ineligible applicant records.

IV. Architectural Decisions

Decision	Selected Approach	Rationale
Application architecture	Layered modular application	Simplifies development and maintenance.
Framework	SvelteKit and TypeScript	Aligns with the development team's planned technology stack.
Data Management	Relational SQL database	Maintains consistency among related records.
Eligibility screening	Rule-based evaluation	Supports transparent application of OLA-approved criteria.
Document management	Protected file storage	Separates uploaded documents from structured database records.
Case triage	Specialization- and workload-based assignment	Facilitates appropriate distribution of legal matters.

Deployment	Centralized infrastructure	UP Supports the project's resource constraints.

B. Content (Data) Design

1. Class Diagrams

LINGAP's classes are organized into three primary functional subsystems: Applicant and Application Management, Scheduling and Consultation Management, and Case Management and Triage.

Applicant and Application Management

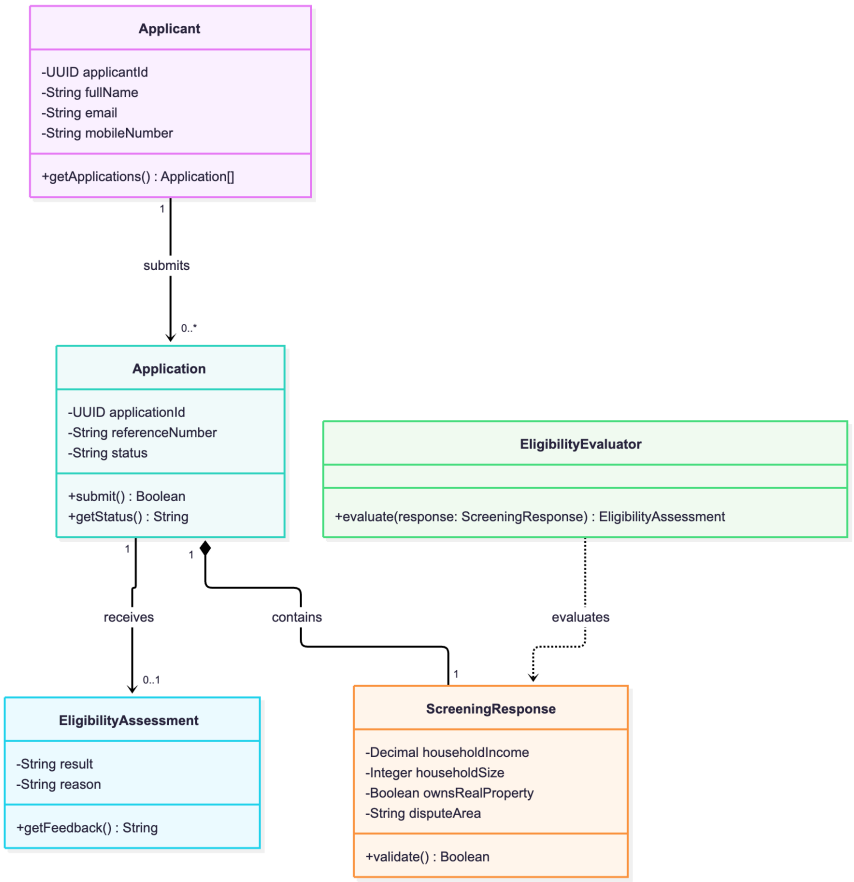


Figure 2.2. Applicant and Application Management class diagram.

An applicant submits applications containing screening responses. The eligibility evaluator assesses these responses using OLA-approved criteria and produces a preliminary eligibility result.

Scheduling and Consultation Management

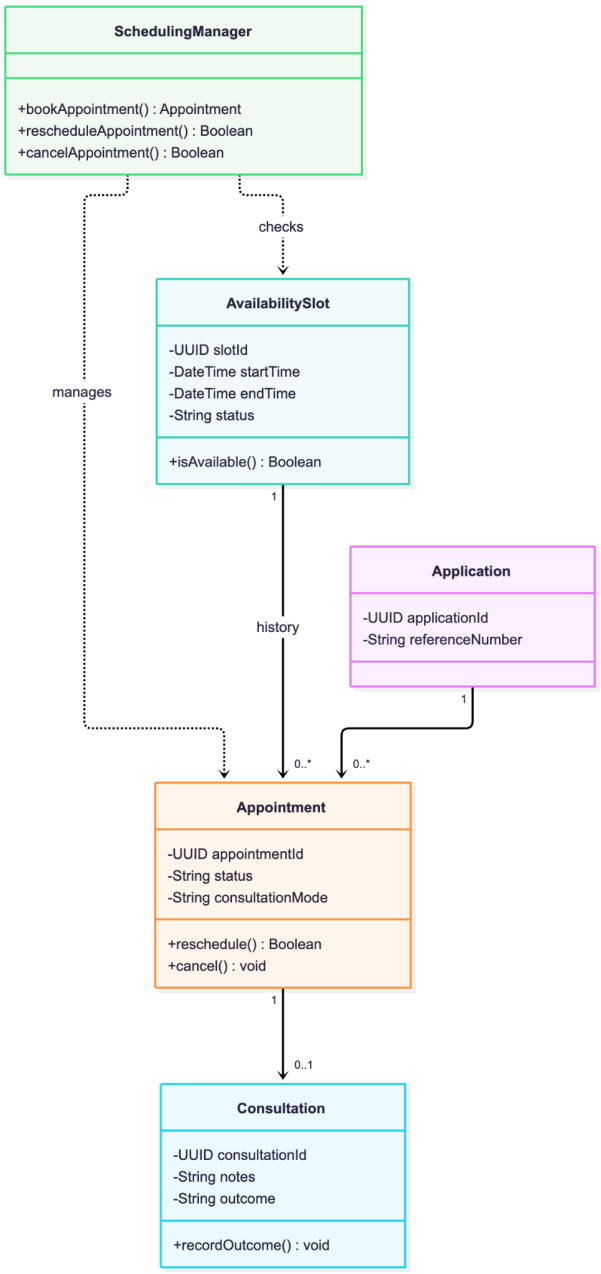


Figure 2.3. Scheduling and Consultation Management class diagram.

The scheduling manager handles appointment reservations, rescheduling, and cancellations based on available consultation slots. Consultation records maintain the outcomes of completed appointments.

Case Management and Triage

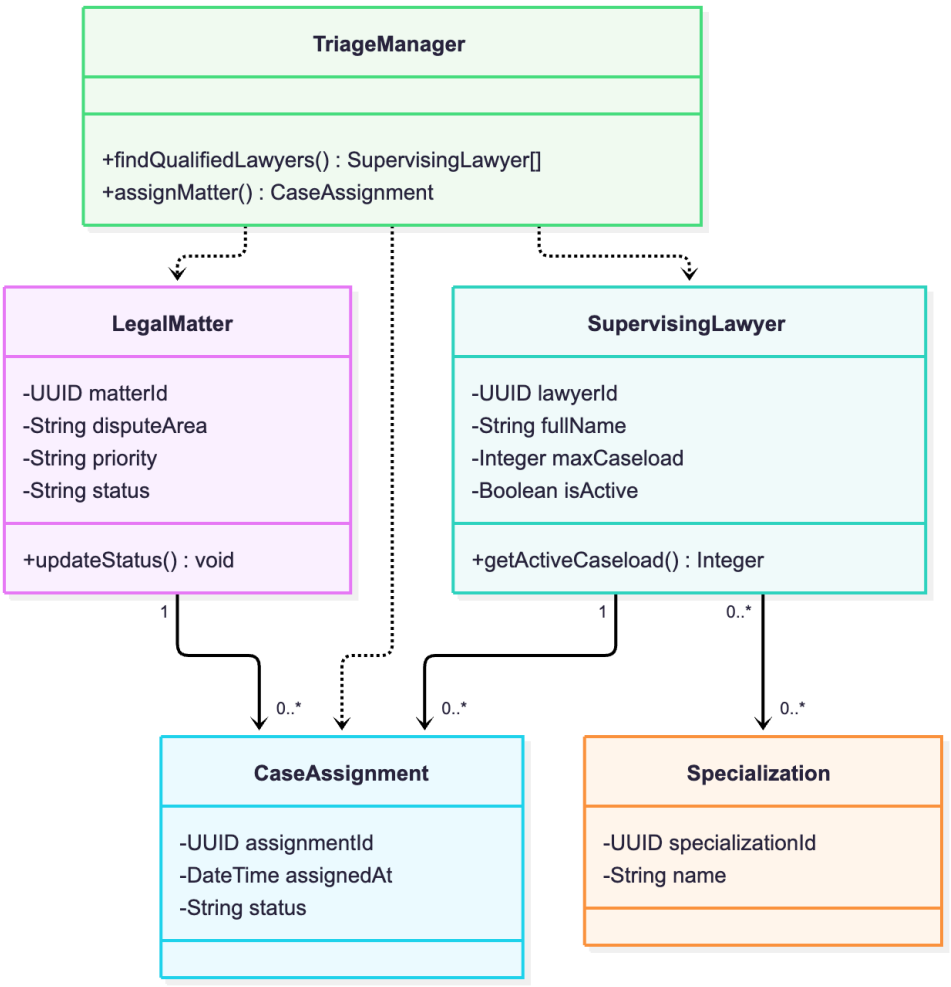


Figure 2.4. Case Management and Triage class diagram.

Legal matters are assigned to supervising lawyers according to their specializations and active caseloads. Assignment records preserve the history of case allocations and reassignment.

2. Entity-Relationship Diagrams

LINGAP uses a relational database to maintain consistent relationships among applicants, applications, screening results, appointments, documents, and legal matters.

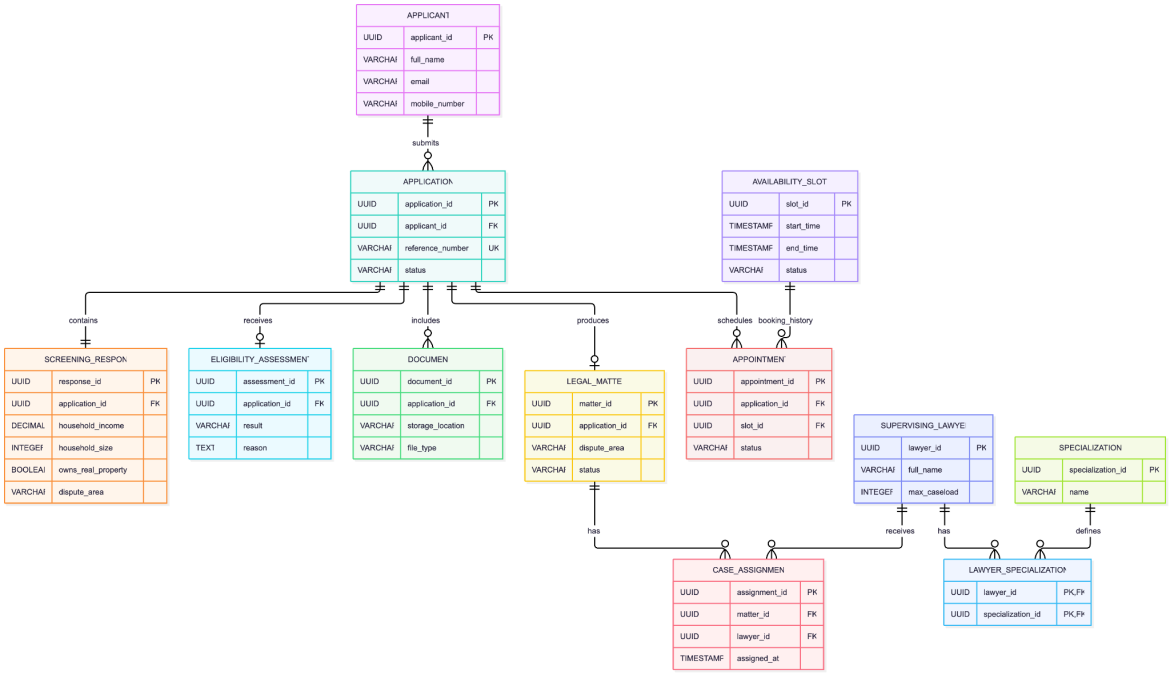


Figure 2.5. Logical Entity-Relationship Diagram of LINGAP.

Primary and foreign keys preserve referential integrity, while unique application reference numbers support application tracking. Document files are maintained separately from relational records, with their metadata stored in the database.

3. Data Flow Diagrams

Level 0 - Context Diagram

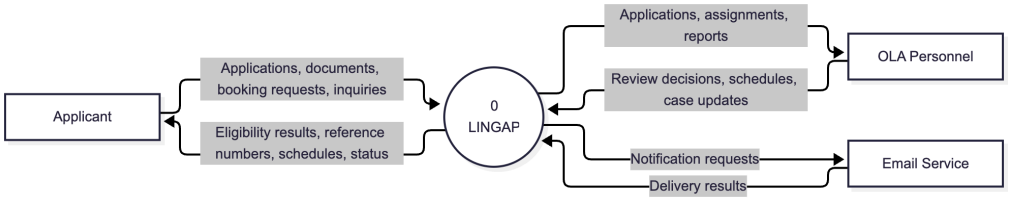


Figure 2.6. Level 0 DFD of LINGAP.

LINGAP receives application information and requests from applicants, administrative inputs from OLA personnel, and delivery responses from an external email service. It returns screening outcomes, application information, case records, and notifications.

Level 1 - Major System Processes

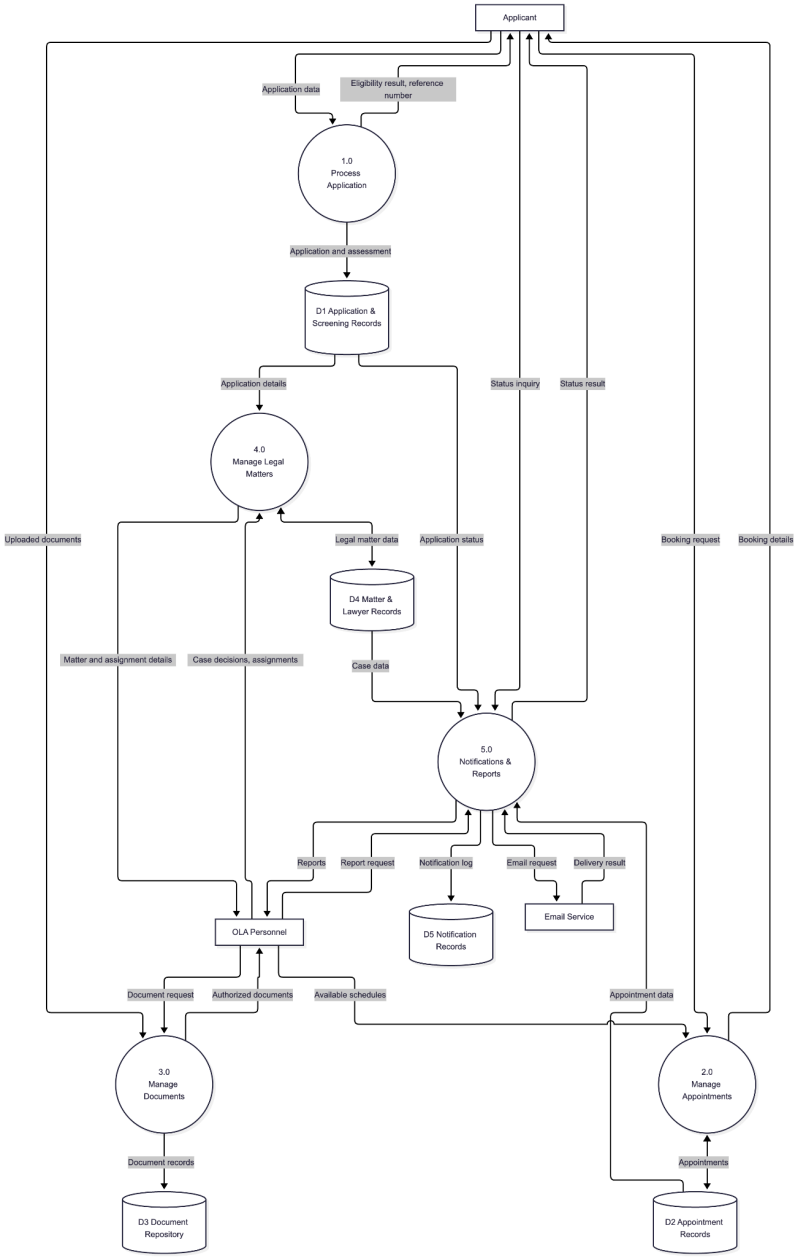


Figure 2.7. Level 1 DFD of LINGAP.

The system processes applications, manages appointments and documents, maintains legal matters, and generates notifications and reports through five interconnected processes.

Level 2 - Application Processing

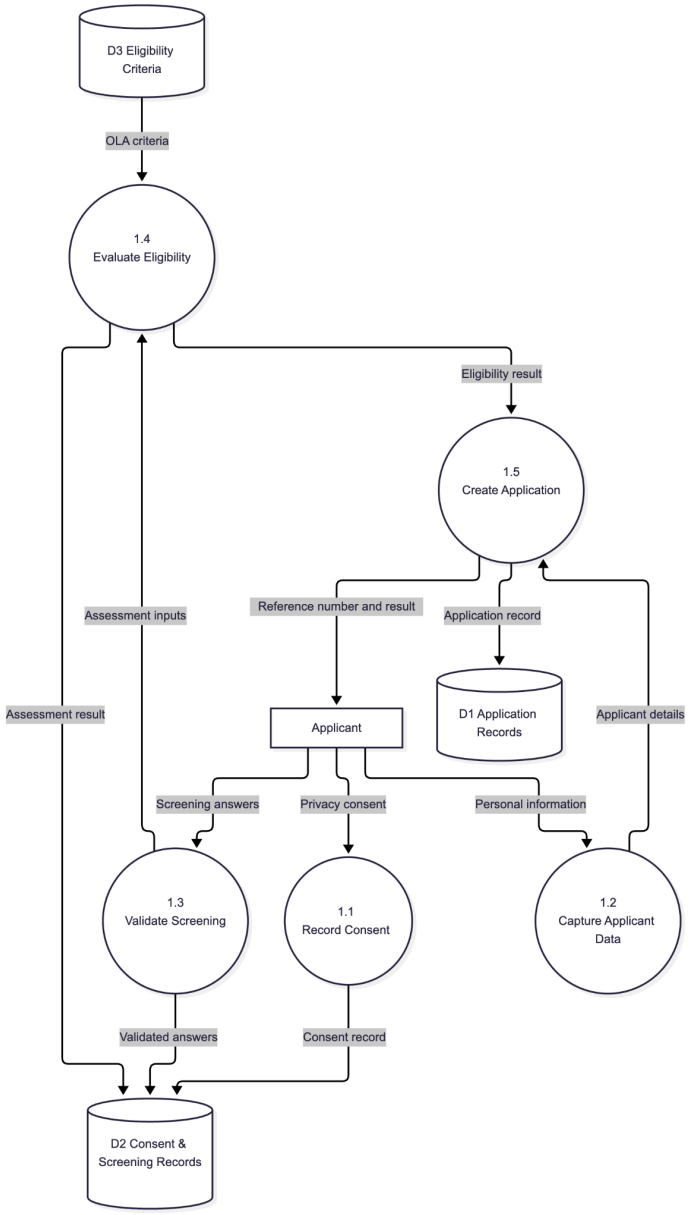


Figure 2.8. Level 2 DFD of LINGAP

Application processing begins with privacy consent and applicant information collection. Screening responses are validated and evaluated against OLA-approved eligibility criteria. The system then generates an application record, unique reference number, and preliminary eligibility result for the applicant.

The stored application information supports subsequent document submission, appointment scheduling, and legal matter processing.

C. Component Level-Designs

LINGAP has three modules (Application, Scheduling, Case Management & Triage) plus shared services. Each module hides its internals behind a small interface and talks to the others only through IDs and those interfaces.

Component	Owner	Provided interface	Depends on
Application	Buton	IApplicationService, IEligibilityEvaluator, IStatusLookup	IReferenceNumberGenerator, INotificationService
Scheduling	Pulbosa	ISchedulingService, ISlotQuery	IApplicationLookup, INotificationService
Case Management & Triage	Rebollido	ICaseService, ITriageService, ITriageStrategy	ICaseloadQuery, INotificationService
Shared services	Pulbosa (integration)	IDocumentStore, INotificationService, IKpiService, IAuthorizationService	Database, file storage, email relay

Dependencies

- Flow: Application → Documents → Scheduling → Triage. A submission must be screened before it can be booked or assigned. Every module sends events to Notification.
- Sharing: all modules read the same central database and the same authorization check. Each module writes only to its own tables.
- Constrained: booking a slot and assigning a lawyer must be mutually exclusive, so two applicants cannot take the last seat and a lawyer cannot exceed the caseload limit. Both are enforced with a database transaction.

COTS and reused components

Component	Use
SvelteKit, TypeScript, Tailwind	Web front end and server routes
SQL database	Central repository with transactions
osTicket (optional)	Ticket numbers and communication logs, called through an adapter so LINGAP still works without it
Google OAuth	Staff sign-in
Email relay (SMTP)	Transactional notifications

Design concepts applied

- Open-Closed Principle: new eligibility criteria and triage policies are added as new classes behind IEligibilityRule and ITriageStrategy, without changing the evaluator or the triage engine.
- Dependency Inversion: services depend on interfaces (repository, email channel), not on the database driver or mail library.
- High cohesion, low coupling: each class does one job (e.g. the reference number service only issues numbers). Modules exchange IDs, not shared objects.
- Information hiding: thresholds, scoring and file paths are hidden from the UI.

1. Algorithms

Eligibility stops at the first failed check. Triage assigns only among lawyers who still have capacity.

Eligibility rules (decision table). Thresholds are loaded from an OLA-approved configuration table, so criteria can change without code changes.

Condition / Action	R1	R2	R3	R4	R5
Consent given	F	T	T	T	T
Dispute area in scope	–	F	T	T	T
Income ≤ threshold for household size	–	–	F	T	T
Owns real property	–	–	–	T	F
Result	Block	Ineligible (scope)	Ineligible (income)	Needs review	Eligible

Triage rule. Among active lawyers with the matching specialization and spare capacity, pick the lowest load ratio (active cases ÷ maximum). Ties go to the lawyer assigned least recently. If nobody qualifies, the case is marked "pending manual routing" and the administrator is notified.

Supporting rules.

- Booking: a slot is re-checked inside a transaction. If it was just taken, the applicant sees the next open times. Rescheduling is cancel plus book in one transaction.
- Reference number: OLA-YYYY-NNNN from a per-year counter, with a unique constraint.
- Purge: a daily job deletes ineligible applications and their documents after 30 days and logs counts only.

2. Data Structures

Criteria: correctness and data integrity first, flexibility for OLA to change rules second, simplicity for a student team third. OLA's volume is small, so database-native structures are preferred over custom in-memory ones.

Structure	Used for	Why
Lookup table (household size → income threshold)	Eligibility	O(1) lookup, editable as configuration
Ordered array of rule objects	Eligibility	Cheap checks run first; new rules just added to the list
Hash map (dispute area → specialization)	Triage	O(1) mapping, easy to edit
Array of candidate lawyers, linear scan	Triage	Few lawyers, so no heap or tree needed
Indexed slot table (date, time, capacity, booked count)	Scheduling	Fast date queries, safe under transactions
Form step graph (step → next-step condition)	Skip logic	Questions are data, not nested if-else
Dictionary keyed by message and language (EN/FIL)	Bilingual UI	Language toggle keeps entered data
FIFO outbox table with retry count	Notifications	Survives restarts and retries failed emails

D. Interface Elements / Usability Design

Users are applicants (often novices, many on phones or the kiosk), LSPs and lawyers (frequent users), and administrators (report readers). The forms and reports below are part of the interface.

Form / report	Format
Pre-screening form	Step-by-step wizard (consent, identification, legal concern, indigency), one topic per screen, progress indicator ("Hakbang 2 ng 4"), draft saved between steps
Eligibility result page	Plain-language outcome. Ineligible: reason, alternative legal aid resources, 30-day deletion note. Eligible: upload and booking buttons
Confirmation slip (screen, email, kiosk printout)	Large reference number, result, appointment date, time and mode, documents to bring, office address, how to check status
Notification emails	Subject with reference number, short body in the applicant's language
KPI report (admin)	Date-range filter, cards for pre-screenings, % pre-screened before visit, deflections, average intake time, charts by dispute area and intake channel, targets beside actuals, CSV or PDF export

How the design handles the Section 11.4.3 issues

Issue	Design response
Response time	About 2 seconds per page with low variability. Progress bar for uploads. Eligibility is computed in one server request
Help	Helper text under fields, plain-language tooltips for legal terms, FAQ, Contact OLA link on every page
Error handling	Plain words, a fix, no blame, icon plus colour. Example: "Mukhang kulang ang numero ng cellphone. Pakilagay ang 11 digit, halimbawa 09171234567."
Menu and command labels	Verb-first, bilingual, and the same label always does the same thing (e.g. "Triage →")
Accessibility	WCAG 2.1 AA target: keyboard use, visible focus, 4.5:1 contrast, labelled inputs, kiosk buttons at least 48 px
Internationalization	EN/FIL toggle at any step without losing data, longer Filipino text allowed for, amounts in PHP

1. Interface Design: Menus and Links

Public portal: the header on every page has Home, an EN | FIL toggle, Simulan ang Pre-Screening and Alamin ang Status ng Aplikasyon. Wizard pages add a progress stepper with Back and Next. The footer links to the Privacy Notice, FAQ and Contact OLA. Kiosk mode uses the same pages with larger buttons and an automatic reset after inactivity.

Administrative portal: a left sidebar, with items shown by role (hidden items are also blocked by the server).

Menu item	LSP	Supervising lawyer	OLA admin
Intake & Triage Dashboard	Yes	Yes	Yes
Appointments and My Cases	Yes	Yes	Yes
Supervising Lawyers	View	View	Manage
Notification Outbox Log	No	No	Yes
Analytics & KPI Dashboard	No	Summary	Yes
Settings (rules, thresholds, retention)	No	No	Yes

2. Aesthetic Design: Layout and Graphics

Element	Specification
Layout	Single column on mobile; centred content on desktop. Public: header, progress stepper, one task card, footer. Admin: left sidebar, top bar, content area
Colours	Maroon #7B1113 (primary actions), forest green #014421 (confirmations), off-white #F8F7F4 background, text #1F2937, error #B42318
Typography	Sans-serif stack, 16 px base (18 px on